



Entwicklung eines Modellfahrzeugs mit Fahrerassistenzfunktionen

Studienarbeit T3_3100

für die Prüfung zum

Bachelor of Science

des Studiengangs Informationstechnik

an der Dualen Hochschule Baden-Württemberg Karlsruhe

von

Leander Gantert & Luca Müller

31. August 2026

Bearbeitungszeitraum
Matrikelnummer, Kurs
Betreuer der DHBW

XX.XX.2025 - XX.XX.2026
3854248, TINF23B3
Dipl.-Ing. (FH) Stefan Lehmann

Zusammenfassung

Das Thema dieser Bachelorarbeit ist die Entwicklung eines Prototyps für eine neue Softwarearchitektur. Diese Architektur basiert auf dem LIN-Bussystem und soll die Kommunikation zwischen Slaves und Mastern überarbeiten. Ziel dieser Arbeit ist es, eine...

Diese Bachelorarbeit behandelt die Entwicklung eines elektronischen Systems zur Temperaturbegrenzung von Strahlungsheizkörpern mithilfe von Thermoelementen. Ziel dieser Arbeit ist es, ein kostengünstigeres und flexibleres System zu entwickeln, das die bislang verwendeten mechanischen Temperaturbegrenzer ersetzt. Hintergrund ist die zunehmende Marktanforderung nach sensorbasierten Lösungen sowie die Notwendigkeit, den Oil-Ignition-Test der Norm UL 858 zu bestehen, der zukünftig auch für Strahlungsheizkörper verpflichtend sein könnte.

Der Kern der Arbeit liegt in der Auswertung von Thermoelementen zur Temperaturbegrenzung der Strahlungsheizkörper. Die Arbeit umfasst die Entwicklung der hierfür notwendigen Hardware sowie die Programmierung der Software für den Mikrocontroller. Zusätzlich wird eine PC-Anwendung entwickelt, die den Systemstatus der entwickelten Steuerung visualisiert und eine Konfiguration der Temperaturbegrenzung ermöglicht.

Abschließend werden Funktionstests des entwickelten Systems durchgeführt, gefolgt von einer Bewertung der Wirtschaftlichkeit des neuen Systems.

Abstract

The scope of this bachelor thesis is the development of an electronic system for temperature limitation of radiant heating elements using thermocouples. The aim of this thesis is to develop a more cost-effective and flexible system to replace the mechanical temperature limiters currently in use. The motivation for this work is the increasing market demand for sensor-based solutions and the need to pass the Oil-Ignition Test of the UL 858 standard, which could also apply to radiant heaters in the future.

The focus of this thesis is measuring the voltage from thermocouples to limit the temperature of the radiant heating elements. Furthermore, the thesis includes the development of the necessary hardware and the development of the microcontroller software. Additionally, a desktop application is developed to visualize the system status of the developed control system and to configure the temperature limits.

Finally, functional tests of the developed system will be conducted, followed by an assessment of the economic efficiency of the new system.

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit T3_3100 mit dem Thema: *Entwicklung eines Modellfahrzeugs mit Fahrerassistenzfunktionen* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Karlsruhe, 31. August 2026

A handwritten signature in black ink, appearing to read 'L. Müller'.

Leander Gantert & Luca Müller

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	III
Listings	III
Abkürzungsverzeichnis	IV
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Ziel dieser Arbeit	2
2 Hardware	3
2.1 Ultraschallsensor	3
2.1.1 Funktionsweise von Ultraschallsensoren	3
2.1.2 Ultraschallsensoren als Frühwarnsystem	3
2.2 Raspberry Pi	4
2.2.1 Aufbau des Raspberry Pi	4
2.2.2 Raspberry Pi als Steuergerät	4
2.3 Motoren	5
2.4 Raspberry Pi Camera Module	6
2.5 Stromversorgung	6
2.6 Gehäuse	7
3 Softwarearchitektur	8
3.1 Übersicht und Aufbau der Architektur	8
3.2 Programmiersprachen	8
3.2.1 Python	8
3.2.2 Weboberfläche	8
3.2.3 Verwendete Bibliotheken und Frameworks	8
3.3 Verwendete Technologien und Tools	9
3.3.1 Entwicklungsumgebung	9

3.3.2	Versionsverwaltung mit Git	9
3.3.3	Protokolle und Frameworks	10
3.4	Softwarekomponenten und deren Kommunikation	10
3.4.1	GUI-Anwendung zur Steuerung	10
3.4.2	Backend-Anwendung zur Steuerung der Hardware	10
3.4.3	Kommunikation zwischen GUI und Backend über gRPC	10
3.5	Implementierte Features	10
3.5.1	Notbremsung durch Hinderniserkennung	11
3.5.2	Schilderkennung	11
3.5.3	Follow the line	11
3.5.4	Fernsteuerung	11
3.5.5	etc.	11
4	Implementierung und Integration	12
4.1	Hardware	12
4.2	Software	12
4.2.1	Notbremsung	12
4.2.2	Schilderkennung	13
4.2.3	Follow the line	13
4.2.4	Fernsteuerung	13
4.2.5	etc.	13
5	Testing und Qualitätssicherung	14
5.1	Teststrategie für die Überarbeitung	14
5.2	Definition der Testfälle	14
5.3	Testdurchführung zur Funktionalitätsprüfung	14
5.4	Evaluation der Testergebnisse	14
6	Fazit und Ausblick	15
	Literaturverzeichnis	V

Abbildungsverzeichnis

2.1	Funktionsweise Ultraschallsensor	3
2.2	3D-gedruckte Halterung als Gehäuse	7

Tabellenverzeichnis

Listings

Abkürzungsverzeichnis

GPIO	General Purpose Input/Output
gRPC	gRPC Remote Procedure Calls
GUI	Graphical User Interface (dt. Grafische Benutzeroberfläche)
IDE	Integrated Development Environment (dt. Integrierte Entwicklungsumgebung)
LAN	Local Area Network
LiPo	Lithium-Polymer
PCIe	Peripheral Component Interconnect Express
PWM	Pulsweitenmodulation
USB	Universal Serial Bus

1 Einleitung

1.1 Motivation

In den letzten Jahren hat die Entwicklung von Fahrassistenzsystemen und autonomen Fahren erheblich zugenommen und ist mittlerweile ein zentrales Thema in der Automobilindustrie. Für viele Autofahrer sind diese Technologien bereits zu einem unverzichtbaren Bestandteil ihres Fahrerlebnisses geworden.

Fahrassistenzsysteme bieten eine Vielzahl an Vorteilen. Sie erhöhen die Sicherheit im Straßenverkehr, indem sie den Fahrer in kritischen Situationen unterstützen und Unfälle verhindern können. Zudem tragen sie zur Reduzierung der Belastung des Fahrers bei, da sie repetitive Aufgaben übernehmen können und so die Ermüdung verringern. Mithilfe von Sensoren und Kamerasystemen können Fahrassistenzsysteme die Umgebung des Fahrzeugs überwachen und auf potenzielle Gefahren reagieren. Dies ermöglicht es dem Fahrer, sich auf andere Aspekte des Fahrens zu konzentrieren und gleichzeitig ein höheres Maß an Sicherheit zu gewährleisten.

1.2 Problemstellung

Da diese Systeme oft eine hohe Komplexität aufweisen und in Gefahrensituationen fehlerfrei funktionieren müssen, ist ihre Entwicklung und Implementierung eine große Herausforderung. Bevor solche Fahrassistenzsysteme in realen Fahrzeugen implementiert werden können, ist es wichtig, diese in kontrollierten Umgebungen ausgiebig zu testen. Doch selbst in solchen Testumgebungen können Fehler zu hohen Kosten durch Schäden an Hardware und noch schlimmer zu Gefährdung Anderer führen. Daher ist es sinnvoll, solche Systeme zunächst in einem kleineren Maßstab zu entwickeln und zu testen.

1.3 Ziel dieser Arbeit

Ziel dieser Arbeit ist die Entwicklung eines kleinen ferngesteuerten Fahrzeuges und die Implementierung einiger grundlegender Fahrassistenzsysteme. Dabei soll das Fahrzeug in der Lage sein eine Notbremsung durchzuführen, einer Linie zu folgen und Schilder zu erkennen und darauf zu reagieren.

Das Fahrzeug soll über eine Weboberfläche gesteuert und überwacht werden können. Hierzu sollen Sensordaten und Kamerabilder in Echtzeit übertragen werden, um dem Benutzer eine umfassende Kontrolle über das Fahrzeug zu ermöglichen.

Die Fernsteuerung soll noch weiter zur Sicherheit und Benutzerfreundlichkeit während der Vorführung und des Testens dieser Fahrassistenzsysteme dienen.

2 Hardware

In diesem Kapitel wird der Aufbau der Hardware und die Funktionsweise der einzelnen Komponenten beschrieben.

2.1 Ultraschallsensor

2.1.1 Funktionsweise von Ultraschallsensoren

Ein Ultraschallsensor nutzt hochfrequente Schallwellen, um die Entfernung zu einem Objekt zu messen. Der Sensor besteht aus einem Sender, sowie einem Empfänger. Der Sender sendet Schallwellen aus, die von einem Objekt reflektiert werden und nach deren Rückkehr vom Empfänger aufgenommen werden. Daraufhin wird die Zeit gemessen, die die Schallwellen benötigt haben, um zum Objekt und zurück zum Sensor zu gelangen. Anhand dieser Zeit und der bekannten Schallgeschwindigkeit kann die Entfernung zum Objekt berechnet werden.

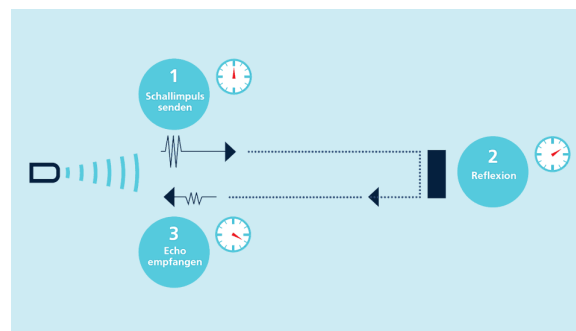


Abb. 2.1 Funktionsweise Ultraschallsensor

2.1.2 Ultraschallsensoren als Frühwarnsystem

In diesem Projekt werden Ultraschallsensoren als Frühwarnsystem zur Hinderniserkennung eingesetzt. Die Sensoren werden so positioniert, dass sie den Bereich vor dem Fahrzeug überwachen können. Wenn ein Hindernis erkannt wird und die Gefahr einer

Kollision besteht, kann automatisch eine Notbremsung eingeleitet werden, um diese zu verhindern.

Ultraschallsensoren sind für diesen Anwendungsfall besonders geeignet, da sie zuverlässig Entfernungen messen können, selbst wenn die Sichtverhältnisse schlecht sind, beispielsweise bei Dunkelheit, Staub oder Nebel. Zudem sind sie kostengünstig und einfach zu integrieren, was sie zu einer idealen Wahl für die Hinderniserkennung in diesem Projekt macht.

(Eine weitere Einsatzmöglichkeit für Ultraschallsensoren ist eine Einparkhilfe, die dem Benutzer visuelle oder akustische Signale gibt, wenn das Fahrzeug während des Einparkens zu nah an ein Hindernis kommt.)

2.2 Raspberry Pi

2.2.1 Aufbau des Raspberry Pi

Ein Raspberry Pi ist ein Einplatinencomputer, der eine Brücke zwischen Hard- und Software bildet. Er verbindet einige der Vorteile eines herkömmlichen Computers und eines Mikrocontrollers. Zum einen besitzt er konfigurierbare GPIO¹ Pins und Anschlüsse für Versorgungsspannung und Erdung, wodurch externe Hardwarekomponenten direkt an den Raspberry Pi angeschlossen werden können. Zum anderen ist er leistungstärker als ein Mikrocontroller. Das neueste Modell ist ausgestattet mit bis zu 16 Gigabytes an RAM, einem Arm-basierten 2,4 GHz Prozessor und einigen Schnittstellen wie beispielsweise einem LAN² Port, einer PCIe³ Schnittstelle und sowohl USB⁴-A als auch USB-C Ports.

2.2.2 Raspberry Pi als Steuergerät

Als Steuergerät für das Fahrzeug wird ein Raspberry Pi verwendet. Der Raspberry Pi verfügt über ausreichend Leistung und Schnittstellen, um die Anforderungen des Projekts zu erfüllen. Ein Mikrocontroller wurde aufgrund der benötigten Rechenleistung

¹ General Purpose Input/Output

² Local Area Network

³ Peripheral Component Interconnect Express

⁴ Universal Serial Bus

und geplanten Features wie der Bildverarbeitung nicht in Betracht gezogen.

Der Raspberry Pi ermöglicht die Ansteuerung der Motoren mithilfe der GPIO Pins, die Verarbeitung der Sensordaten und die Kommunikation mit der Weboberfläche. Zudem bietet er die Möglichkeit, verschiedene Bibliotheken und Frameworks zu nutzen, um die Implementierung

2.3 Motoren

Als Motoren werden zwei Gleichstrommotoren verwendet, welche direkt an die beiden Räder des Fahrzeugs montiert werden. Die Motoren verfügen über eine interne Getriebeübersetzung, welche die Drehzahl reduziert und das Drehmoment erhöht. Dies ermöglicht eine präzisere Steuerung des Fahrzeugs und ein im Vergleich zu einem Motor ohne Getriebe höheres Drehmoment, was insbesondere bei niedrigen Geschwindigkeiten von Vorteil ist.

Um das Fahrzeug zu bewegen, werden die Motoren jeweils in die gleiche Richtung bewegt. Das Lenken erfolgt durch die Ansteuerung der Motoren in unterschiedliche Richtungen. Wenn beispielsweise der linke Motor vorwärts und der rechte Motor rückwärts angetrieben wird, dreht sich das Fahrzeug nach rechts und umgekehrt. Durch die Möglichkeit, die Motoren unabhängig voneinander zu steuern, kann das Fahrzeug präzise manövriert werden, was insbesondere bei der Umsetzung von Fahrassistenzsystemen von Vorteil ist.

Wie aus dem Datenblatt zu entnehmen ist (einfügen), haben die Motoren eine Nennspannung von 6 Volt und eine maximale Stromaufnahme von 1,8 Ampere. Um die Motoren zu steuern wird ein *TB6612FNG* Motortreiber verwendet. Dieser besitzt intern eine H-Brücke, welche die Ansteuerung der Motoren in beide Richtungen ermöglicht. Zudem bietet er die Möglichkeit, die Geschwindigkeit der Motoren durch PWM¹ zu steuern.

¹ Pulsweitenmodulation

2.4 Raspberry Pi Camera Module

Das Raspberry Pi Camera Module ist eine Kamera, die speziell für die Verwendung mit dem Raspberry Pi entwickelt wurde. Sie bietet eine Auflösung von bis zu 12 Megapixeln und einen Blickwinkel von 160 Grad, was sie ideal für die Erfassung von Bildern und Videos in diesem Projekt macht.

2.5 Stromversorgung

Zur Stromversorgung des Fahrzeugs wird ein LiPo¹ Akku verwendet. Auf Grund der hohen Energiedichte ist ein LiPo Akku eine solide Wahl für den Anwendungszweck (Hier vergleich mit lipefo akkus). Jedoch gibt es auch einige Nachteile. Die Zellchemie des Akkus ist sehr empfindlich gegenüber Tiefenentladung sowie mechanischen Beschädigungen, weshalb eine sorgfältige Handhabung und Überwachung der Akkus notwendig ist.

Der Akku verfügt über eine Kapazität von 5000 mAh und eine Nennspannung von 7,4 Volt, was ausreichend ist, um das Fahrzeug für mindestens eine Stunde mit Strom zu versorgen. Da die Motoren eine Nennspannung von 6 Volt und der Raspberry Pi eine Nennspannung von 5 Volt haben, werden mehrere Spannungsregler verwendet, um die benötigten Spannungen für die verschiedenen Komponenten bereitzustellen. Ein Step-Down Spannungsregler wird verwendet, um die Spannung von 7,4 Volt auf 5 Volt zu reduzieren, um den Raspberry Pi mit Strom zu versorgen. Zwei weitere Step-Down Spannungsregler werden verwendet, um die Spannung von 7,4 Volt auf 6 Volt zu reduzieren, um die Motoren mit Strom zu versorgen.

Da ein einzelner Spannungswandler nur maximal 5 Ampere ausgeben kann, ist es notwendig, mehrere Spannungswandler zu verwenden, da der Raspberry Pi alleine bereits unter Volllast 5 Ampere aufnehmen kann.

¹ Lithium-Polymer

2.6 Gehäuse

Als Gehäuse für das Fahrzeug wird lediglich eine 3D-gedruckte Halterung verwendet auf der alle Komponenten montiert sind. Diese werden nicht weiter geschützt, da das Fahrzeug nur in einer kontrollierten Umgebung eingesetzt wird. Außerdem soll so das Gewicht des Fahrzeugs möglichst gering gehalten werden und die Komponenten sollen leicht zugänglich sein, um Wartungsarbeiten und Anpassungen zu erleichtern. Die Halterung wird so konstruiert, dass alle Komponenten sicher befestigt sind und gleichzeitig ausreichend Platz für die Verkabelung und Belüftung vorhanden ist. Das Design der Halterung ermöglicht es, die Komponenten einfach zu montieren und bei Bedarf auszutauschen oder zu erweitern. In Abbildung 2.2 ist die 3D-gedruckte Halterung zu sehen, auf der alle Komponenten des Fahrzeugs montiert sind.

Abb. 2.2 3D-gedruckte Halterung als Gehäuse

3 Softwarearchitektur

3.1 Übersicht und Aufbau der Architektur

3.2 Programmiersprachen

Als Programmiersprache für die Softwareentwicklung wird Python verwendet, da diese eine gute Kompatibilität mit dem Raspberry Pi bietet und eine Vielzahl an Bibliotheken und Frameworks bietet.

3.2.1 Python

Python ist eine interpretierte, objektorientierte Programmiersprache, die gute Lesbarkeit und einfache Syntax bietet. Durch die umfangreiche Standardbibliothek und die Vielzahl an Drittanbieter-Bibliotheken eignet sich Python besonders gut für sowohl die Entwicklung von Anwendungen mit Hardware-Interaktion als auch für die Implementierung von Video-Übertragung. Dadurch ist Python ideal für die Entwicklung der Software für das ferngesteuerte Fahrzeug.

3.2.2 Weboberfläche

3.2.3 Verwendete Bibliotheken und Frameworks

Für die Entwicklung der Software werden verschiedene Bibliotheken und Frameworks verwendet, um die geplanten Funktionalitäten umzusetzen und die Entwicklung zu erleichtern. Dazu gehören beispielsweise:

- **OpenCV:** Eine Bibliothek für die Bildverarbeitung, die für die Implementierung der Schilderkennung und Hinderniserkennung verwendet wird.

- **RPi.GPIO:** Eine Bibliothek zur Steuerung der GPIO-Pins des Raspberry Pi, die für die Ansteuerung der Motoren und Sensoren verwendet wird.
- **gRPC:** Ein Framework für die Remote Procedure Call (RPC) Kommunikation, das für die Kommunikation zwischen der GUI-Anwendung und der Backend-Anwendung genutzt wird.

3.3 Verwendete Technologien und Tools

In diesem Abschnitt werden die Technologien und Tools beschrieben, die für die Entwicklung der Software verwendet werden. Dazu gehören beispielsweise die Entwicklungsumgebung und Versionsverwaltungssysteme.

3.3.1 Entwicklungsumgebung

Als IDE¹ wird Visual Studio Code verwendet, da diese eine gute Unterstützung für Python bietet und eine große Auswahl an Erweiterungen zur Verfügung stellt, die die Entwicklung erleichtern. Zudem ist Visual Studio Code plattformübergreifend verfügbar und kann somit sowohl auf einem Laptop als auch auf dem Raspberry Pi genutzt werden, unabhängig vom Betriebssystem.

3.3.2 Versionsverwaltung mit Git

Für die Versionsverwaltung des Quellcodes wird GitHub verwendet. GitHub bietet umfangreiche Funktionen zur Zusammenarbeit mit anderen Entwicklern, um Quellcodeänderungen zu verfolgen und zu verwalten. Durch die Nutzung von GitHub können mehrere Entwickler gleichzeitig am Projekt arbeiten und Änderungen einfach zusammengeführt und nachverfolgt werden. (mehr über Git/GitHub? Brauchen wir diesen Abschnitt überhaupt?)

¹ Integrated Development Environment (dt. Integrierte Entwicklungsumgebung)

3.3.3 Protokolle und Frameworks

Für die Kommunikation zwischen den Softwarekomponenten werden verschiedene Protokolle verwendet, um eine zuverlässige Datenübertragung zu gewährleisten.

gRPC Remote Procedure Call

Das gRPC¹ Framework wird für die Kommunikation zwischen der GUI²-Anwendung und der Backend-Anwendung genutzt. (nutzt RPC, Protocol Buffer?, vorteile von gRPC?)

HTTP

Protokolle als eigene section?

3.4 Softwarekomponenten und deren Kommunikation

3.4.1 GUI-Anwendung zur Steuerung

3.4.2 Backend-Anwendung zur Steuerung der Hardware

3.4.3 Kommunikation zwischen GUI und Backend über gRPC

3.5 Implementierte Features

In diesem Abschnitt werden die für das Fahrzeug geplanten Features beschrieben.

¹ gRPC Remote Procedure Calls

² Graphical User Interface (dt. Grafische Benutzeroberfläche)

3.5.1 Notbremsung durch Hinderniserkennung

Das Fahrzeug soll in der Lage sein, eine Notbremsung durchzuführen, wenn es ein Hindernis erkennt. Hierzu werden Ultraschallsensoren verwendet, die den Bereich vor dem Fahrzeug überwachen und die Entfernung zu Hindernissen messen. Wird ein Hindernis erkannt, das eine Kollision verursachen könnte, wird automatisch eine Notbremsung eingeleitet. Um zu ermitteln, ob eine Kollision wahrscheinlich ist, wird die gemessene Entfernung unter Berücksichtigung der aktuellen Geschwindigkeit des Fahrzeugs ausgewertet.

3.5.2 Schilderkennung

3.5.3 Follow the line

3.5.4 Fernsteuerung

3.5.5 etc.

Grundlagen: - tools - Programmiersprache: - Bibliotheken - Frameworks - Technologien
- IDEs? - Protokolle - gRPC - HTTP - etc.

- GUI-Anwendung zur Steuerung - Backend-Anwendung zur Steuerung der Hardware
- Kommunikation zwischen GUI und Backend über gRPC - Features: - Schilderkennung
- Hinderniserkennung - Follow the line - Fernsteuerung - etc.

4 Implementierung und Integration

4.1 Hardware

4.2 Software

4.2.1 Notbremsung

Eine Notbremsung wird durchgeführt, wenn ein Hindernis erkannt wird. Die gemessene Entfernung zu diesem Hindernis wird unter Berücksichtigung der aktuellen Geschwindigkeit des Fahrzeugs ausgewertet. Dadurch kann ermittelt werden, ob eine Kollision wahrscheinlich ist.

Die Auswertung der gemessenen Entfernung erfolgt durch die Berechnung des Bremswegs, der anhand der aktuellen Geschwindigkeit berechnet wird. Hierzu wird sowohl die Formel für eine normale Bremsung als auch für eine Notbremsung herangezogen. Die Formel für die Berechnung des Bremswegs bei einer normalen Bremsung lautet:

$$s = \frac{v}{10} \cdot \frac{v}{10} \quad (4.1)$$

wobei s der Bremsweg in Metern und v die Geschwindigkeit in km/h ist.

Für eine Notbremsung wird die folgende Formel verwendet:

$$s = \frac{v}{10} \cdot \frac{v}{10} \cdot 0.5 \quad (4.2)$$

Dabei wird der Bremsweg um 50 % reduziert, um die schnellere Reaktion und stärkere Betätigung des Bremspedals bei einer Notbremsung zu berücksichtigen [1].

Basierend auf diesen Formeln wird eine weitere Formel für die tatsächliche Berechnung der Notbremsung erstellt:

$$s = \frac{v}{10} \cdot \frac{v}{10} \cdot 0.75 \quad (4.3)$$

Hierbei wird der Bremsweg lediglich um 25 % reduziert, da es sich zwar um eine Notbremsung handelt, aber dadurch eine mögliche Bremsverzögerung berücksichtigt wird.

4.2.2 Schilderkennung

4.2.3 Follow the line

4.2.4 Fernsteuerung

4.2.5 etc.

5 Testing und Qualitätssicherung

5.1 Teststrategie für die Überarbeitung

5.2 Definition der Testfälle

5.3 Testdurchführung zur Funktionalitätsprüfung

5.4 Evaluation der Testergebnisse

6 Fazit und Ausblick

Wir haben n richtig geiles Auto gebaut und Lehmann fand die Ausarbeitung und das Gefährt so krass, dass er uns eine 0,8 gegeben hat für unser Werk.

Literaturverzeichnis

- [1] **Bremsweg berechnen.** Jan. 2026. URL: <https://www.adac.de/verkehr/rund-um-den-fuehrerschein/erwerb/bremsweg-berechnen/> (Abruf: 05.02.2026).